Mathematical Foundations of Transaction Fraud-Risk Modeling: A BankSim Case Study

Derivations, Decision Theory, Validation Evidence, and Deployment
A Companion to the Complete CRISP-DM Project

Morad Ahmadnasab

Intelligent Transaction Risk & Fraud Detection Platform — A BankSim Case Study

Technical Report — October 2026

Abstract

This technical report presents a reproducible BankSim case study of supervised, imbalanced binary transaction-fraud classification and subsequent selective intervention. Following the six-phase CRISP-DM framework, we examine time-indexed synthetic transactions, fit a Logistic Regression baseline and six Random Forest configurations using transaction amount and category, and compare discrimination, probability quality, and operational consequences. The methodological decisions are linked to the predictor types, the binary target, chronological validation, class imbalance, and the distinction between estimated risk and chosen action. RF-3 was selected for its highest validation Average Precision (0.807737), although RF-2 obtained lower validation Brier score and log loss. With model and two action thresholds frozen before testing, RF-3 achieved test Average Precision 0.804028 and ROC-AUC 0.992570. The resulting Approve/Review/Escalate policy routed 880 of 1,080 labeled fraud transactions (81.48%) to intervention while assigning 1,726 of 100,117 transactions (1.72%) to Review or Escalate. We provide mathematical derivations, notebook-linked experimental evidence, and a reproducible inference workflow. These findings concern synthetic BankSim data and do not demonstrate real-world calibration, prevention effectiveness, financial savings, or production readiness.

Contents

1 Modeling objectives and experimental design **2**
- 1.1 From a business question to a statistical problem . . . 2
- 1.2 Statistical learning, estimation, and generalization . . . 2
- 1.3 Problem taxonomy, objectives, and methodological scope . . . 3
- 1.4 CRISP-DM: decisions, rationale, and evidence . . . 3

2 Logistic regression baseline: mathematics before implementation **4**
- 2.1 The dataset, chronological split, and why order matters . . . 4
- 2.2 Constructing predictors: scaling and category indicators . . . 5
- 2.3 Logistic regression: why this model, and what is estimated? . . . 6
- 2.4 Maximum likelihood, cross-entropy, regularization, and optimization . . . 6

3 Validation probabilities and ranking performance **8**
- 3.1 From a fitted model to a score . . . 8
- 3.2 ROC-AUC: definition and meaning . . . 9
- 3.3 Precision–recall curve and Average Precision . . . 9

4 Classification at the reference threshold 0.5 **10**
4.1 The decision rule and confusion matrix . . . 10
4.2 Deriving the evaluation metrics . . . 11

5 Threshold sensitivity, score distributions, and calibration **12**
5.1 Why threshold selection changes the operating point . . . 12
5.2 What calibration means . . . 13

6 Three-action policies and statistical decision theory **14**
6.1 Prediction and action are different mathematical objects . . . 14
6.2 Bayes risk and its assumptions . . . 14
6.3 Worked Bayes-risk example and active decision boundaries . . . 14
6.4 Capacity constraints and ranking . . . 15
6.5 Frozen validation policy . . . 15

7 Random Forests, complexity control, and model comparison **15**
7.1 Tree and forest probability estimators . . . 15
7.2 How classification trees select splits . . . 16
7.3 Observed validation comparison . . . 16
7.4 Why RF-3 was selected, and what was traded off . . . 17

8 Final evaluation on the reserved chronological test partition **18**
8.1 Protocol and predictive results . . . 18
8.2 Action-by-outcome analysis . . . 18
8.3 Fixed thresholds versus realized workload . . . 19
8.4 Hypothetical economic sensitivity . . . 19
8.5 What the test results can and cannot support . . . 20

9 Reproducible inference, monitoring, and limitations **20**
9.1 Monitoring framework . . . 21
9.2 Educational synthesis . . . 21

10 Principal findings, contributions, limitations, and conclusions **21**
10.1 What the experiments established . . . 21
10.2 What this project contributes . . . 21
10.3 Limits and next steps . . . 21

11 Questions for mathematical understanding and independent study **22**

1 Modeling objectives and experimental design

Project correspondence: modeling notebook Sections 4.1–4.2. The section numbers in this companion are independent of notebook numbering.

1.1 From a business question to a statistical problem

Project methodology and data source. This project follows the six phases of the Cross-Industry Standard Process for Data Mining (CRISP-DM): business understanding, data understanding, data preparation, modeling, evaluation, and deployment [3]. The phase-specific objectives, decisions, and deliverables are documented in `docs/01_business_understanding.md` through `docs/06_deployment.md`; the present companion focuses on the mathematical methods and verified experimental findings. BankSim is a synthetic bank-payment simulation dataset developed for fraud-detection research [8]. Its generated labels and distributions must not be assumed representative of operational banking data.

A transaction arrives with recorded amount A and category G. Its unknown eventual fraud status is the response $Y \in \{0, 1\}$, where $Y = 1$ means fraud. In the notation of *An Introduction to Statistical Learning with Applications in Python* (ISLP), $X = (A, G)$ is the predictor vector, and the object of interest is

$$p(X) = \mathbb{P}(Y = 1 \mid X), \qquad \hat{p}(X) \text{ is an estimate of } p(X). \tag{1}$$

This is *binary supervised classification*, because the observed response has two labels. It is not a three-class classification task: approve, review, and escalate are prospective *actions* produced by a downstream policy, not observed BankSim training labels. The distinction between estimating an unknown conditional probability and choosing an action is central to statistical decision theory [1].

1.2 Statistical learning, estimation, and generalization

Let $\mathcal{D}_{\text{train}} = \{(X_i, Y_i)\}_{i=1}^{n}$, with $n = 396{,}332$. A learning algorithm uses this sample to construct $\hat{p}$. Good in-sample fit is not the same as good future prediction: the relevant question is how $\hat{p}$ behaves on transactions not used to fit it. We therefore distinguish training (parameter estimation), validation (development and comparison), and test (final, untouched evaluation) [7, Chs. 2,5].

Partition	Steps	Transactions	Fraud	Prevalence
Training	0–125	396,332	5,040	1.271661%
Validation	126–152	98,194	1,080	1.099864%
Test	153–179	100,117	1,080	1.078738%

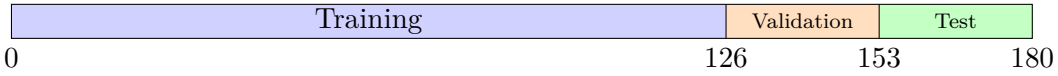


Figure 1: Chronological simulation-step partitions. Labels at 126, 153, and 180 denote partition boundaries; the final observed step is 179.

As shown in Figure 1, the fractions 70/15/15 arise from dividing the 180 simulation steps into 126/27/27 steps (exactly 70%/15%/15%); they refer to *simulation steps*, not transaction counts. Chronological partitioning prevents training on later steps when evaluating earlier ones; it does not establish that BankSim mimics real-time banking. Every simulation step has exactly 40 fraud cases, an artificial regularity. *No code is required at this design stage*: these are design decisions preceding implementation.

1.3 Problem taxonomy, objectives, and methodological scope

The observed response is a *labeled binary outcome*; hence the learning task is supervised classification, not unsupervised anomaly detection. Estimating $p(x) = \mathbb{P}(Y = 1 \mid X = x)$ is a related probabilistic prediction task, whereas selecting an intervention is a *decision problem* with different potential errors and operational costs [7, 1]. The data are mixed-type tabular records (numerical amount and categorical merchant category), heavily imbalanced, and indexed by simulation step. They are not a verified sample of live financial transactions [8]. The three actions are *not* three target classes: they are post-prediction policy choices.

The statistical aim is to estimate scores that rank and, as far as the data permit, quantify fraud risk. The operational aim is to route higher-risk transactions for human review or escalation with limited intervention workload. These aims are related but not equivalent: good ranking does not guarantee calibrated probabilities, and a high-performing classifier does not itself specify an optimal cost-sensitive decision policy [5, 1]. Our contribution is an integrated, documented implementation and evaluation of established methods on one synthetic dataset; we do not propose a novel classifier or claim deployment validation.

1.4 CRISP-DM: decisions, rationale, and evidence

CRISP-DM supplies a six-phase, iterative project structure [3], *not* a prescription to choose Logistic Regression, Random Forests, or a 70/15/15 split. The following project-specific decisions connect the data and objectives to the methods actually implemented.

1. **Business understanding.** We formulated the task as prioritizing transactions for intervention, not simply maximizing overall classification accuracy. Rare fraud and unequal potential consequences motivate discrimination, workload, and decision-oriented measures (Sections 3–6). The operational costs are illustrative assumptions, not observed losses.

2. **Data understanding.** We examined transaction counts, label prevalence, amount, category, and simulation steps. The complete synthetic sample contains 594,643 transactions and 7,200 fraud labels. Low prevalence motivates precision–recall analysis alongside ROC analysis [12, 4].

3. **Data preparation.** We selected `amount` (numerical) and `category` (categorical) as a deliberately limited, auditable predictor set. One-hot encoding converts categories to numeric indicators; training-fitted standardization supports the Logistic Regression baseline. The chronological step split (126/27/27 of 180 steps) avoids training on later simulated steps when assessing earlier predictions. Its proportions are an experimental choice, not a universal CRISP-DM rule (Section 2).

4. **Modeling.** We first fitted regularized Logistic Regression as a transparent probabilistic baseline for binary labels, then compared Random Forests to accommodate nonlinear and interaction effects without manually specifying them [7, 6, 2]. Six forest configurations explore complexity control; this is a limited candidate comparison, not an exhaustive search (Section 7).

5. **Evaluation.** We used validation Average Precision as the primary selection criterion, with ROC-AUC, Brier score, log loss, and reliability diagrams as complementary evidence. The selected RF-3 and two thresholds were fixed before the reserved chronological test. Action counts and assumed-cost scenarios assess consequences of the *policy*, not causal fraud prevention (Sections 5–8).

6. **Deployment.** We saved the preprocessing-plus-model pipeline and a separate JSON policy, and checked standalone inference against notebook predictions on five examples. This demonstrates limited reproducibility, not production certification (Section 9).

These phases were documented in the corresponding `docs/01_business_understanding.md` through `docs/06_deployment.md` files. Each choice is conditional on the observed data and intended use: a different dataset, feature-availability constraint, intervention cost structure, or time-dependence may justify different methods and metrics.

2 Logistic regression baseline: mathematics before implementation

2.1 The dataset, chronological split, and why order matters

Mathematical preparation. Let $S_i \in \{0, \ldots, 179\}$ denote the simulation step of transaction i. Define index sets

$$I_{\text{tr}} = \{i : S_i \leq 125\},$$
$$I_{\text{val}} = \{i : 126 \leq S_i \leq 152\},$$
$$I_{\text{te}} = \{i : S_i \geq 153\}.$$

They are pairwise disjoint, their union is the entire sample, and their step ranges are ordered. No stratification or random shuffling is appropriate for reproducing our approved experiment. We use only the training predictor matrix X_{tr} and labels Y_{tr} to fit model parameters. *Implementation:* the following is the first part of the original modeling notebook cell.

```
from pathlib import Path

import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# Load the original BankSim dataset
DATA_PATH = Path("../data/raw/bs140513_032310.csv")
df = pd.read_csv(DATA_PATH)

# Define the approved baseline configuration
BASELINE_FEATURES = ["amount", "category"]
TARGET = "fraud"
TIME_COLUMN = "step"

# Reproduce the approved chronological partitions
train_df = df.loc[df[TIME_COLUMN] <= 125].copy()

valid_df = df.loc[
    (df[TIME_COLUMN] >= 126)
    & (df[TIME_COLUMN] <= 152)
].copy()

test_df = df.loc[df[TIME_COLUMN] >= 153].copy()

# Verify consistency with Data Preparation
assert len(df) == 594643
assert len(train_df) == 396332
assert len(valid_df) == 98194
assert len(test_df) == 100117

assert train_df[TIME_COLUMN].max() < valid_df[TIME_COLUMN].min()
assert valid_df[TIME_COLUMN].max() < test_df[TIME_COLUMN].min()

# Separate training predictors and target
X_train = train_df[BASELINE_FEATURES].copy()
y_train = train_df[TARGET].copy()
```

Interpretation. The assertions check sample counts and chronological separation. They are consistency checks, not proofs that every conceivable leakage mechanism has been eliminated. Raw data remain unchanged. The relative path in the listing is resolved from the modeling notebook working directory (typically `notebooks/`), not from the directory containing this LaTeX source.

2.2 Constructing predictors: scaling and category indicators

Choice of preprocessing. The predictor `amount` is numerical, while `category` is nominal. One-hot encoding preserves category distinctions without imposing a fictitious numeric ordering; standardization puts the amount coefficient on a training-derived scale and helps numerical optimization of the regularized Logistic Regression baseline. Tree-based models do not generally require feature scaling, so this preprocessing is motivated primarily by the baseline and the common pipeline, not by a universal Random Forest requirement.

Mathematical preparation. We need a numerical design vector to fit logistic regression. For training amounts $A_1, \ldots, A_n$, define the training mean and population standard deviation

$$\bar{A}_{\mathrm{tr}} = \frac{1}{n}\sum_{i=1}^{n} A_i, \qquad s_{\mathrm{tr}} = \left[\frac{1}{n}\sum_{i=1}^{n}(A_i - \bar{A}_{\mathrm{tr}})^2\right]^{1/2}, \qquad Z_i = \frac{A_i - \bar{A}_{\mathrm{tr}}}{s_{\mathrm{tr}}}. \tag{2}$$

Scikit-learn's `StandardScaler` uses denominator n, not the unbiased sample-variance denominator $n-1$ [16]. Assuming $s_{\mathrm{tr}} > 0$, we have the following elementary guarantee.

Proposition 2.1. *The transformed training values satisfy* $n^{-1}\sum_i Z_i = 0$ *and* $n^{-1}\sum_i Z_i^2 = 1$.

Proof. The first follows from $\sum_i (A_i - \bar{A}_{\mathrm{tr}}) = 0$. The second follows immediately from the definition of s_{tr}^2. □

For validation and test, use *the same* training mean and standard deviation; otherwise evaluation would use information from the future partition. Their transformed means and standard deviations need not equal zero and one.

The fitted preprocessing pipeline reports 16 transformed predictors: one scaled amount and 15 category indicators. This is evidence for 15 observed training categories; a direct reproducibility check is `train_df["category"].nunique()` (expected result: 15). Let $g_1, \ldots, g_{15}$ be these training categories, and set $D_{ik} = \mathbf{1}\{G_i = g_k\}$. The transformed row is

$$X_i = (Z_i, D_{i1}, \ldots, D_{i,15})^\top \in \mathbb{R}^{16}. \tag{3}$$

There is exactly one active indicator per row when the category is among those seen in training. `handle_unknown="ignore"` produces all-zero category indicators for an unseen category [15]; none occurred in our validation or test partitions.

An important identifiability subtlety. With an intercept and all 15 indicators, $\sum_k D_{ik} = 1$ makes the unregularized design linearly dependent. Coefficient values are not individually identifiable without constraints. Penalizing category coefficients by an ℓ_2 term selects a representation, but the coefficient of one category alone should not be read as an unadjusted, uniquely identified category effect. For a change from category g_j to g_k, the change in fitted log-odds is $\hat{\beta}_{k+1} - \hat{\beta}_{j+1}$ (assuming β_1 corresponds to amount).

2.3 Logistic regression: why this model, and what is estimated?

Model choice. Logistic Regression provides a conventional, comparatively interpretable baseline for binary labels, with a smooth probabilistic score and a convex penalized fitting objective. Its additive linear-log-odds specification may miss nonlinear amount effects and category interactions, motivating the subsequent Random Forest comparison (Section 7). This is a modeling rationale, not a claim that Logistic Regression is universally preferable.

Mathematical preparation. Linear regression of Y on X could return values outside $[0, 1]$. Logistic regression instead models the *log-odds* as a linear function [7, Ch. 4]:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 Z + \sum_{k=1}^{15} \beta_{k+1} D_k =: \eta(X). \tag{4}$$

Define the sigmoid (logistic) function $\sigma(u) = 1/(1+e^{-u})$ for $u \in \mathbb{R}$. Solving algebraically for p gives

$$p(X) = \frac{\exp(\eta(X))}{1+\exp(\eta(X))} = \frac{1}{1+\exp(-\eta(X))} =: \sigma(\eta(X)). \tag{5}$$

Since $e^{-\eta} > 0$, the denominator in (5) is strictly greater than one; hence $0 < p(X) < 1$. This verifies that the logistic link always produces a value in the open unit interval.

Odds and odds ratios. For $0 < p < 1$, the *odds* are $p/(1-p)$, and their logarithm is the *log-odds*. The odds ratio compares the odds at two predictor settings.

Proposition 2.2 (Odds-ratio interpretation)**.** *Holding all other design coordinates fixed, a feasible one-unit increase in a continuous predictor X_j multiplies the modeled odds by e^{β_j}.*

Proof. Exponentiating (4) gives odds $\exp(\eta(X))$; adding one to X_j adds β_j to η and multiplies odds by e^{β_j}. □

For Z, a one-unit increase means one *training* standard deviation in amount. Switching categorical levels changes two indicator coordinates and thus requires comparing their coefficients.

2.4 Maximum likelihood, cross-entropy, regularization, and optimization

Mathematical preparation. Under a conditional Bernoulli model, $Y_i \mid X_i \sim \text{Bernoulli}(p_i)$, with $p_i = \sigma(\eta_i)$, and assume the labels are conditionally independent given the observed predictors. The Bernoulli mass function assigns probability $p_i^{Y_i}(1-p_i)^{1-Y_i}$ to the observed label Y_i. Multiplying these terms gives the likelihood L, and taking its logarithm gives the log-likelihood ℓ:

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} p_i^{Y_i}(1-p_i)^{1-Y_i}, \tag{6}$$

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} [Y_i \log p_i + (1-Y_i)\log(1-p_i)]. \tag{7}$$

Maximizing ℓ is equivalent to minimizing the negative log-likelihood, also called *binary cross-entropy*:

$$J(\boldsymbol{\beta}) = \sum_{i=1}^{n} [-Y_i \log p_i - (1-Y_i)\log(1-p_i)]. \tag{8}$$

Here $\boldsymbol{\beta} = (\beta_0, \beta_1, \ldots, \beta_{16})^\top$. Let $\tilde{X}$ be the $n \times 17$ matrix whose first column is ones and remaining columns are the transformed features. To derive the derivatives, write the ith loss as

$\log(1+e^{\eta_i}) - Y_i\eta_i$, where $\eta_i = \tilde{x}_i^\top \boldsymbol{\beta}$. Its first derivative with respect to η_i is $p_i - Y_i$ and its second derivative is $p_i(1-p_i)$, since $\partial p_i/\partial \eta_i = p_i(1-p_i)$. By the chain rule, summing over observations yields

$$\nabla J(\boldsymbol{\beta}) = \tilde{X}^\top(\boldsymbol{p} - \boldsymbol{y}), \qquad \nabla^2 J(\boldsymbol{\beta}) = \tilde{X}^\top W \tilde{X}, \quad W = \operatorname{diag}(p_i(1-p_i)). \tag{9}$$

Proposition 2.3 (Convexity of the unpenalized logistic objective)**.** *J is convex in $\boldsymbol{\beta}$.*

Proof. For any $v \in \mathbb{R}^{17}$ (including $v = 0$),

$$v^\top \nabla^2 J(\boldsymbol{\beta}) v = (\tilde{X}v)^\top W (\tilde{X}v) = \sum_{i=1}^{n} p_i(1-p_i)(\tilde{x}_i^\top v)^2 \geq 0.$$

Therefore the Hessian is positive semidefinite at every finite parameter vector, so J is convex. Positive *definiteness* would require a strict inequality for every nonzero v; it does not hold here because the intercept equals the sum of the category-indicator columns. □

The implemented `LogisticRegression` uses L2 regularization. Its settings are `C=1.0` and `class_weight=None`, with the default L-BFGS solver [14, 11].

Conceptually, the fitted coefficients minimize logistic loss plus a positive multiple of $\sum_{j=1}^{16} \beta_j^2/2$, with the intercept unpenalized. In the unweighted binary case, scikit-learn's L-BFGS formulation can be written as

$$\min_{\beta_0,\ldots,\beta_{16}} \frac{1}{n} J(\boldsymbol{\beta}) + \frac{1}{2Cn} \sum_{j=1}^{16} \beta_j^2. \tag{10}$$

This normalization is equivalent to minimizing $J + \frac{1}{2C}\sum_j \beta_j^2$. It matters when comparing textbook regularization parameters λ with software parameter C; do not assume $C = \lambda$. L2 regularization reduces sensitivity to some coefficient instabilities but does not guarantee correct probabilities, prevent all overfitting, or establish generalization.

Proposition 2.4 (Strict convexity of the implemented penalized objective)**.** *For $C > 0$ and a nonempty training sample, the objective in (10) is strictly convex in the intercept and all feature coefficients.*

Proof. Write $\boldsymbol{\beta} = (\beta_0, b)$, with $b \in \mathbb{R}^{16}$. For any nonzero direction $(a, u) \in \mathbb{R} \times \mathbb{R}^{16}$, the Hessian quadratic form is

$$\frac{1}{n}\sum_{i=1}^{n} p_i(1-p_i)(a + X_i^\top u)^2 + \frac{1}{Cn}\|u\|_2^2.$$

If $u \neq 0$, the second term is strictly positive. If $u = 0$, then $a \neq 0$ and the first term is strictly positive because $0 < p_i < 1$. Thus the Hessian is positive definite at every finite parameter vector, implying strict convexity. □

Strict convexity ensures *at most* one minimizer; existence also requires consideration. Here both outcome classes occur in training, and the penalized objective is coercive: the penalty controls b, while the loss diverges as $|\beta_0| \to \infty$ with b bounded. Consequently a unique finite minimizer exists.

Implementation. The next code is the remaining model-construction and training portion of our original modeling notebook cell. All mathematics above precedes its execution.

```
preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", StandardScaler(), ["amount"]),
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            ["category"]
```

```
        )
    ],
    remainder="drop"
)

# Construct the complete estimator pipeline
baseline_model = Pipeline(
    steps=[
        ("preprocessing", preprocessor),
        (
            "classifier",
            LogisticRegression(
                penalty="l2",
                C=1.0,
                class_weight=None,
                max_iter=1000
            )
        )
    ]
)

# Fit preprocessing and classifier on training data only
baseline_model.fit(X_train, y_train)

# Inspect the fitted model
classifier = baseline_model.named_steps["classifier"]
fitted_preprocessor = baseline_model.named_steps["preprocessing"]

print("Training completed successfully.")
print("Model:", type(classifier).__name__)
print("Training observations:", len(X_train))
print("Prepared feature count:",
      len(fitted_preprocessor.get_feature_names_out()))
print("Target classes:", classifier.classes_)
print("Iterations used:", classifier.n_iter_)
print("Maximum iterations:", classifier.max_iter)
print("Coefficient matrix shape:", classifier.coef_.shape)
print("Intercept shape:", classifier.intercept_.shape)
```

Observed output and interpretation. Training completed on 396,332 transactions; there are 16 transformed features, classes $[0, 1]$, coefficient shape $(1, 16)$, intercept shape $(1,)$, and 16 solver iterations out of a maximum of 1,000. These values are consistent with the mathematical model and successful fitting. A low iteration count is *not* a proof of predictive accuracy. The validation and test sets were not used in fitting.

3 Validation probabilities and ranking performance

Why these metrics? With roughly one percent labeled fraud, overall accuracy could remain high for an unhelpful majority-class classifier. We therefore evaluate how well scores order positive cases (ROC-AUC and Average Precision), and separately how useful their numerical probability values are (Brier score, log loss, and reliability diagrams in Section 5). None of these alone fixes an operational action threshold [4, 12, 5].

3.1 From a fitted model to a score

Mathematical preparation. Given a new validation transaction X_i, apply the *training-fitted* scaling and encoding, then calculate $\hat{\eta}_i = \hat{\beta}_0 + \sum_{j=1}^{16} \hat{\beta}_j X_{ij}$ and $\hat{p}_i = \sigma(\hat{\eta}_i)$. Because class labels are $[0, 1]$, we must select the column of `predict_proba` corresponding to class 1. No classification threshold is needed to compare rankings.

3.2 ROC-AUC: definition and meaning

For a threshold t, define $\widehat{Y}_i(t) = \mathbf{1}\{\hat{p}_i \geq t\}$, where fraud ($Y_i = 1$) is the positive class. On the evaluation set, $\mathrm{TP}(t)$ counts actual fraud correctly predicted as fraud (true positives); $\mathrm{FN}(t)$ counts actual fraud predicted as non-fraud (false negatives); $\mathrm{TN}(t)$ counts actual non-fraud correctly predicted as non-fraud (true negatives); and $\mathrm{FP}(t)$ counts actual non-fraud predicted as fraud (false positives). Equivalently:

	$\widehat{Y}(t) = 1$	$\widehat{Y}(t) = 0$
$Y = 1$	$\mathrm{TP}(t)$	$\mathrm{FN}(t)$
$Y = 0$	$\mathrm{FP}(t)$	$\mathrm{TN}(t)$

The true-positive rate (TPR, also called recall or sensitivity) is $\mathrm{TPR}(t) = \mathrm{TP}(t)/(\mathrm{TP}(t) + \mathrm{FN}(t))$ and false-positive rate (FPR, equal to one minus specificity) is $\mathrm{FPR}(t) = \mathrm{FP}(t)/(\mathrm{FP}(t) + \mathrm{TN}(t))$. These ratios assume nonzero denominators. Precision is $\mathrm{TP}(t)/(\mathrm{TP}(t) + \mathrm{FP}(t))$ when defined; it is *not* FPR. The ROC curve plots $\mathrm{TPR}(t)$ versus $\mathrm{FPR}(t)$ over thresholds. ROC-AUC is the area under this curve; equivalently, for independently sampled positive and negative examples, it is the probability that the positive example scores higher, plus half the probability of a tie [4]. ROC-AUC measures discrimination, not calibration.

3.3 Precision–recall curve and Average Precision

Precision and recall at threshold t are

$$\mathrm{Precision}(t) = \frac{\mathrm{TP}(t)}{\mathrm{TP}(t) + \mathrm{FP}(t)}, \qquad \mathrm{Recall}(t) = \frac{\mathrm{TP}(t)}{\mathrm{TP}(t) + \mathrm{FN}(t)}. \tag{11}$$

With ranked score thresholds and successive recall levels R_k, scikit-learn defines *non-interpolated Average Precision* as

$$\mathrm{AP} = \sum_k (R_k - R_{k-1}) P_k, \tag{12}$$

where P_k is precision at the corresponding operating point [13]. This is *not* the trapezoidal area under an interpolated precision–recall curve. With rare positives, precision and the precision–recall curve are sensitive to prevalence; the validation prevalence provides a useful no-skill reference, not a guarantee of a particular random sample's AP [12].

Implementation. The code below is the corresponding modeling notebook cell, now placed *after* the definitions and mathematical interpretation.

```
import numpy as np

from sklearn.metrics import average_precision_score, roc_auc_score

# Separate validation predictors and target
X_valid = valid_df[BASELINE_FEATURES].copy()
y_valid = valid_df[TARGET].copy()

# Estimate fraud probabilities using the fitted pipeline
fraud_class_index = list(
    baseline_model.classes_
).index(1)

valid_probabilities = baseline_model.predict_proba(X_valid)[
    :, fraud_class_index
]

# Calculate ranking metrics
validation_ap = average_precision_score(
    y_valid, valid_probabilities
)
```

```
validation_roc_auc = roc_auc_score(
    y_valid, valid_probabilities
)

validation_prevalence = y_valid.mean()

# Compare AP with the prevalence reference
ap_lift = validation_ap / validation_prevalence

# Verify the probability output
assert len(valid_probabilities) == len(y_valid)
assert np.isfinite(valid_probabilities).all()
assert np.all(
    (valid_probabilities >= 0)
    & (valid_probabilities <= 1)
)

# Report validation results
print("Validation transactions:", len(y_valid))
print("Validation fraud cases:", int(y_valid.sum()))
print(f"Validation fraud prevalence: {validation_prevalence:.6f}")
print(f"Average Precision (AP): {validation_ap:.6f}")
print(f"ROC-AUC: {validation_roc_auc:.6f}")
print(f"AP / prevalence: {ap_lift:.2f}")
print(
    "Predicted fraud probability range:",
    f"[{valid_probabilities.min():.6f}, "
    f"{valid_probabilities.max():.6f}]"
)
```

Observed output. Validation size 98,194; fraud cases 1,080; prevalence 0.010999; AP 0.783930; ROC-AUC 0.992308; AP/prevalence 71.28; predicted score range $[0.000014, 1.000000]$ when rounded to six decimal places.

Interpretation and limits. A ROC-AUC of 0.992308 indicates exceptionally strong *ranking* on these synthetic validation data. AP 0.783930 is far above the prevalence reference. Neither figure means that 78.393% of fraud was caught; neither establishes probability calibration. The displayed maximum of 1.000000 may be a rounded value rather than exactly one. Strong results with only amount and category invite examination of BankSim's synthetic generation mechanisms and future distribution shift.

4 Classification at the reference threshold 0.5

4.1 The decision rule and confusion matrix

Mathematical preparation. We now choose $t = 0.5$ *as a descriptive reference*, not an optimized operating threshold:

$$\widehat{Y}_i = \begin{cases} 1, & \hat{p}_i \geq 0.5, \\ 0, & \hat{p}_i < 0.5. \end{cases} \tag{13}$$

Comparing $\widehat{Y}_i$ to Y_i produces four mutually exclusive counts: TN (actual 0, predicted 0), FP (actual 0, predicted 1), FN (actual 1, predicted 0), and TP (actual 1, predicted 1). The 2×2 confusion matrix is

$$\begin{pmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{pmatrix}, \tag{14}$$

with rows ordered by actual label $0, 1$ and columns by predicted label $0, 1$. It follows from disjointness of the four cases that their sum equals validation sample size, and $\text{TP} + \text{FN}$ equals the number of actual fraud cases.

4.2 Deriving the evaluation metrics

Precision and recall are as in (11). The F1-score is their harmonic mean,

$$\mathrm{F1} = \frac{2\,\mathrm{Precision}\,\mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}} = \frac{2\mathrm{TP}}{2\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}, \tag{15}$$

when denominators are nonzero. Specificity and false-positive rate are

$$\mathrm{Specificity} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}, \qquad \mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{TN} + \mathrm{FP}} = 1 - \mathrm{Specificity}. \tag{16}$$

The fraction of *all* transactions flagged is $(\mathrm{TP} + \mathrm{FP})/N$. A high precision and a low false-positive rate can coexist with many missed fraud cases when the positive class is rare. Threshold changes trade off flagged workload and missed fraud; they do not alter the previously calculated ranking metrics for a fixed set of scores.

Implementation. The following approved notebook cell implements these mathematical definitions on the validation predictions already calculated.

```
import numpy as np

from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)

# Define the conventional reference threshold
REFERENCE_THRESHOLD = 0.5

# Convert validation probabilities into binary predictions
valid_predictions = (
    valid_probabilities >= REFERENCE_THRESHOLD
).astype(int)

# Calculate the confusion matrix
tn, fp, fn, tp = confusion_matrix(
    y_valid,
    valid_predictions,
    labels=[0, 1]
).ravel()

# Calculate classification metrics
precision = precision_score(
    y_valid, valid_predictions, zero_division=0
)

recall = recall_score(
    y_valid, valid_predictions, zero_division=0
)

f1 = f1_score(
    y_valid, valid_predictions, zero_division=0
)

specificity = tn / (tn + fp)
false_positive_rate = fp / (fp + tn)

# Calculate transaction-flagging statistics
flagged_transactions = int(valid_predictions.sum())
flagged_percentage = 100 * flagged_transactions / len(y_valid)

missed_fraud = int(fn)

# Verify consistency of classification counts
assert tn + fp + fn + tp == len(y_valid)
assert tp + fn == int(y_valid.sum())
```

```
assert tp + fp == flagged_transactions

# Report results
print(f"Reference threshold: {REFERENCE_THRESHOLD:.2f}")
print(f"Validation transactions: {len(y_valid):,}")
print()

print("Confusion matrix:")
print(f"True negatives (TN): {tn:,}")
print(f"False positives (FP): {fp:,}")
print(f"False negatives (FN): {fn:,}")
print(f"True positives (TP): {tp:,}")
print()

print("Classification metrics:")
print(f"Precision: {precision:.6f}")
print(f"Recall: {recall:.6f}")
print(f"F1-score: {f1:.6f}")
print(f"Specificity: {specificity:.6f}")
print(f"False-positive rate: {false_positive_rate:.6f}")
print()

print("Transaction flagging:")
print(f"Flagged transactions: {flagged_transactions:,}")
print(f"Flagged percentage: {flagged_percentage:.4f}%")
print(f"Missed fraud cases: {missed_fraud:,}")
```

Observed output. TN = 97,058, FP = 56, FN = 451, TP = 629; N = 98,194. The counts check: 97,058 + 56 + 451 + 629 = 98,194 and 629 + 451 = 1,080.

$$\begin{aligned}
\text{Precision} &= \frac{629}{629+56} = 0.918248,\\
\text{Recall} &= \frac{629}{629+451} = 0.582407,\\
\text{F1} &= \frac{2(629)}{2(629)+56+451} = 0.712748,\\
\text{Specificity} &= \frac{97058}{97058+56} = 0.999423,\\
\text{FPR} &= \frac{56}{97058+56} = 0.000577,\\
\text{Flagging rate} &= \frac{629+56}{98194} \times 100\% = 0.6976\%.
\end{aligned}$$

Interpretation. Among 685 flagged transactions, 629 are fraudulent (high precision), but 451 of the 1,080 actual fraud cases are missed (recall 58.24%). These are observations at the *reference threshold* only. They do not imply that 0.5 is optimal for financial loss, customer experience, or investigation capacity.

5 Threshold sensitivity, score distributions, and calibration

5.1 Why threshold selection changes the operating point

For a score $S = \hat{p}(X)$ and threshold t, the predicted label is $\hat{Y}_t = \mathbf{1}\{S \geq t\}$. Raising t can only shrink the set of flagged transactions, while precision need not change monotonically. In particular,

$$\text{TPR}(t) = \mathbb{P}(S \geq t \mid Y = 1), \quad \text{FPR}(t) = \mathbb{P}(S \geq t \mid Y = 0), \quad \text{Precision}(t) = \mathbb{P}(Y = 1 \mid S \geq t). \tag{17}$$

The threshold 0.5 is a useful reference but has no automatic operational optimality under class imbalance, asymmetric losses, or finite investigation capacity. The notebook's Sections 4.3.3 and 4.3.4 examined threshold sensitivity and risk-score distributions on validation data.

5.2 What calibration means

A perfectly calibrated score S satisfies $\mathbb{E}[Y \mid S = s] = s$ (in the conditional-expectation sense). For bins B_k of scores, a reliability diagram compares

$$\bar{s}_k = \frac{1}{n_k} \sum_{i:S_i \in B_k} S_i, \qquad \bar{y}_k = \frac{1}{n_k} \sum_{i:S_i \in B_k} Y_i. \tag{18}$$

Probability quality and proper scoring rules are treated in [5], while practical calibration of supervised classifiers is examined in [10]. Finite-bin reliability diagrams are descriptive: they depend on bin definitions and sample sizes, and cannot prove calibration at every score. The Brier score $n^{-1} \sum_i (S_i - Y_i)^2$ and log loss $-n^{-1} \sum_i [Y_i \log S_i + (1 - Y_i) \log(1 - S_i)]$ assess probability quality, not just ranking.

Calibration versus discrimination. A model can rank transactions well yet systematically overestimate fraud probabilities. Conversely, a nearly constant predictor at the population prevalence can have a small Brier score on an imbalanced sample but poor ranking power. Calibration bins are sensitive to binning and sample counts, particularly in sparsely populated high-score regions. The notebook diagrams are descriptive; they do not establish perfect calibration [10, 5].

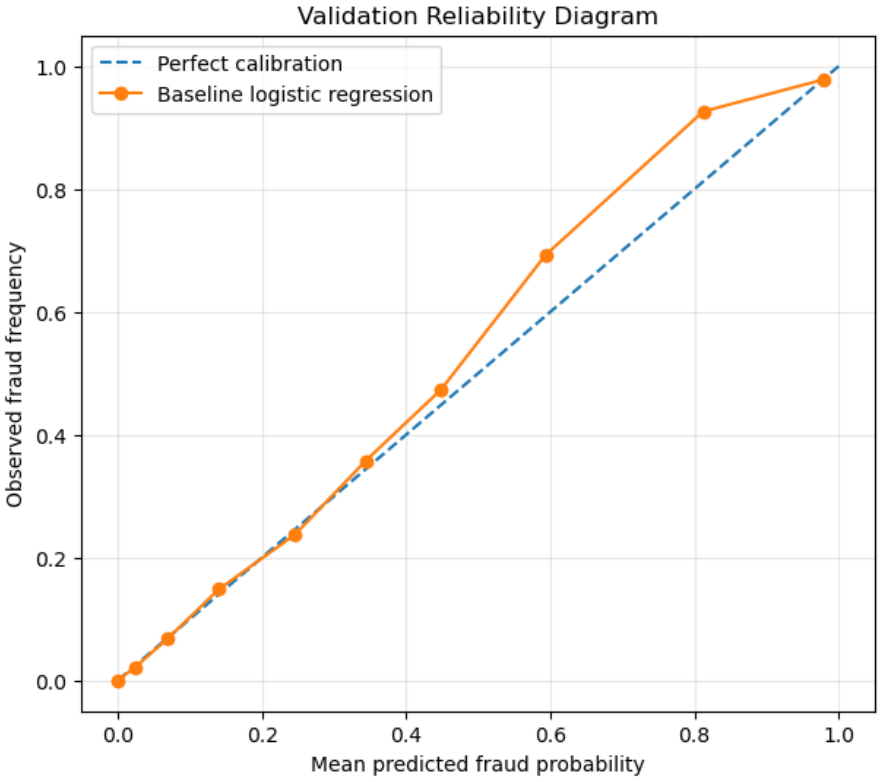


Figure 2: Original validation calibration figure exported from the modeling notebook.

6 Three-action policies and statistical decision theory

6.1 Prediction and action are different mathematical objects

Let $\mathcal{A} = \{A, R, E\}$ denote Approve, Review, and Escalate. The classifier estimates a score $S(X)$; a policy $\delta(S)$ chooses an action. BankSim provides binary fraud labels, not three observed action labels. A two-threshold policy is

$$\delta(s) = \begin{cases} A, & s < \tau_L, \\ R, & \tau_L \le s < \tau_H, \\ E, & s \ge \tau_H. \end{cases} \qquad 0 \le \tau_L < \tau_H \le 1. \tag{19}$$

6.2 Bayes risk and its assumptions

Suppose $p = \mathbb{P}(Y = 1 \mid X)$ is a calibrated conditional probability and $L(a, y)$ is a known loss for action a and true state y. The conditional risk is

$$r(a \mid p) = (1 - p)L(a, 0) + pL(a, 1), \qquad \delta^*(p) \in \arg\min_{a \in \mathcal{A}} r(a \mid p). \tag{20}$$

Each action risk is affine in p; the Bayes decision regions are determined by their lower envelope. When $L(a, 1) - L(a, 0) \neq L(b, 1) - L(b, 0)$, the crossing of two action risks occurs at

$$p_{ab} = \frac{L(b, 0) - L(a, 0)}{[L(a, 1) - L(a, 0)] - [L(b, 1) - L(b, 0)]}. \tag{21}$$

Not every loss matrix yields three nonempty, ordered decision regions: a candidate crossing may lie outside $[0, 1]$, or one action may never minimize conditional risk. Moreover, the model score need not be a calibrated real-world probability. Thus the thresholds used in this project are *validation-derived operational thresholds*, not demonstrated Bayes-optimal cutoffs from known real-world losses.

6.3 Worked Bayes-risk example and active decision boundaries

For illustration only, consider the normalized loss matrix

Action	$L(a, 0)$	$L(a, 1)$
Approve	0	100
Review	2	20
Escalate	12	5

The conditional risks are

$$r(A \mid p) = 100p, \qquad r(R \mid p) = 2 + 18p, \qquad r(E \mid p) = 12 - 7p. \tag{22}$$

The Approve–Review crossing is $p = 2/82 \approx 0.02439$, while the Review–Escalate crossing is $p = 10/25 = 0.4$. The Approve–Escalate crossing is $12/107 \approx 0.11215$, but it is *not* an active Bayes boundary: Review has lower risk there.

Proposition 6.1 (Three nonempty ordered regions in this example)**.** *The Bayes action is Approve for $0 \le p < 2/82$, Review for $2/82 < p < 0.4$, and Escalate for $0.4 < p \le 1$, with ties at the boundaries.*

Proof. Compare the three affine risks on the intervals determined by their pairwise intersections. At $p = 0$, Approve minimizes risk; at $p = 0.1$, Review does; and at $p = 1$, Escalate does. The two active intersections are $2/82$ and 0.4, and the remaining intersection lies strictly above the lower envelope. □

These illustrative Bayes thresholds are *not* the thresholds fitted for the project. The latter were determined by validation-score ranks and operational reference capacities; they do not follow from the example loss matrix.

6.4 Capacity constraints and ranking

With review and escalation capacity limits K_R, K_E, one may formulate

$$\min_{\delta_1,\dots,\delta_n} \sum_{i=1}^{n} r(\delta_i \mid p_i) \quad \text{subject to} \quad \sum_i \mathbf{1}\{\delta_i = R\} \le K_R, \quad \sum_i \mathbf{1}\{\delta_i = E\} \le K_E. \tag{23}$$

A fixed score threshold and a top-K ranking rule are distinct: fixed thresholds permit action counts to vary with the score distribution, whereas top-K fixes the number selected (up to ties and eligibility rules). Their performance should not be conflated.

6.5 Frozen validation policy

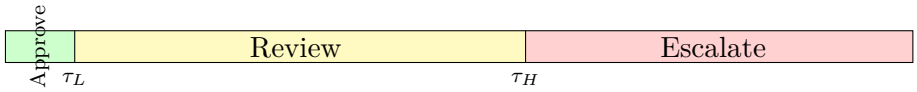


Figure 3: Schematic of the validation-derived three-action policy; interval widths reflect the score thresholds, not observed action frequencies.

After validation development, the selected RF-3 policy used

$$\tau_L = 0.07635097141364822, \qquad \tau_H = 0.5731998416519604. \tag{24}$$

Action	Validation transactions	Percentage
Approve	96,398	98.171%
Review	1,111	1.131%
Escalate	685	0.698%

The percentages are descriptive, rounded values. The thresholds were frozen before test evaluation. In the executed notebook, the upper cutoff was the 685th largest validation RF-3 score and the lower cutoff the 1,796th largest score. The target counts were realized on validation; ties at the cutoff can in general alter exact counts. On test, the unchanged cutoffs yielded different workloads.

7 Random Forests, complexity control, and model comparison

Why this comparison? Logistic Regression assumes an additive linear structure on the log-odds scale. Random Forests offer a flexible alternative that can learn nonlinear split patterns and interactions from the same tabular predictors. Depth and minimum-leaf constraints were varied to examine overfitting and probability quality, not to claim an exhaustive hyperparameter search [2, 6].

7.1 Tree and forest probability estimators

Following the Random Forest framework of [2], for a classification tree with terminal leaf $\ell(x)$, the empirical leaf probability is

$$\hat{p}_{\text{tree}}(x) = \frac{\sum_{i:X_i \in \ell(x)} Y_i}{\#\{i : X_i \in \ell(x)\}}. \tag{25}$$

For a forest of B trees, the usual class-probability prediction is the average of individual tree predictions:

$$\hat{p}_{\mathrm{RF}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{p}_b(x). \tag{26}$$

Bootstrap sampling and random feature subsets encourage diversity among trees [2]. Maximum depth and minimum leaf size control complexity and can influence both discrimination and probability quality. Unlike a convex logistic-regression objective, random-forest fitting is a greedy, randomized ensemble procedure; there is no single globally optimized differentiable objective of the same form.

7.2 How classification trees select splits

For a node t containing n_t labeled examples, let $\hat{\pi}_{t,k}$ denote the empirical fraction in class $k \in \{0, 1\}$. The Gini impurity is

$$G(t) = 1 - \sum_{k=0}^{1}\hat{\pi}_{t,k}^2 = 2\hat{\pi}_{t,1}(1 - \hat{\pi}_{t,1}). \tag{27}$$

A candidate split into left and right children is evaluated through the weighted decrease

$$\Delta G = G(t) - \frac{n_L}{n_t}G(t_L) - \frac{n_R}{n_t}G(t_R). \tag{28}$$

A greedy tree chooses a permitted split maximizing ΔG (subject to its feature and sample constraints). For example, a node with 10 fraud and 90 non-fraud observations has $G = 2(0.1)(0.9) = 0.18$. If a proposed split produces one child with 8 fraud and 12 non-fraud observations and another with 2 fraud and 78 non-fraud observations, the weighted post-split impurity is $0.2(0.48) + 0.8(0.04875) = 0.135$, yielding $\Delta G = 0.045$.

In the fitted forest, each tree is trained with bootstrap sampling and considers a random subset of candidate predictors at each split. The `max_depth` limit restricts path length; `min_samples_leaf` prevents very small terminal nodes. These are regularization mechanisms, not guarantees of calibrated probabilities or external generalization.

7.3 Observed validation comparison

The following table and figures compare the candidate models on the same chronological validation partition. The charts show discrete model configurations rather than points on a continuous parameter scale.

Model	AP ↑	ROC-AUC ↑	Brier ↓	Log loss ↓
Logistic regression	0.783930	0.992308	0.004312	0.016985
RF-0 (unrestricted)	0.708612	0.908137	0.005720	0.084423
RF-1	0.792198	0.991686	0.004142	0.016680
RF-2	0.805055	0.992544	**0.004003**	**0.015751**
RF-3 (selected)	**0.807737**	**0.992853**	0.004038	0.015968
RF-4	0.803650	0.992634	0.004108	0.016025
RF-5	0.797432	0.992811	0.004258	0.016543

Table 1: Actual chronological-validation metrics from the completed modeling notebook.

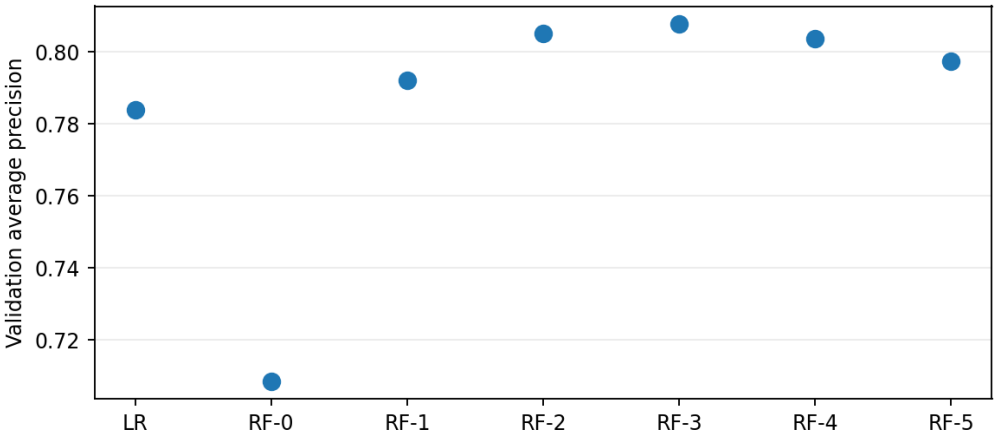


Figure 4: Discrete-model comparison of Average Precision; reconstructed from the notebook's exact validation summary values.

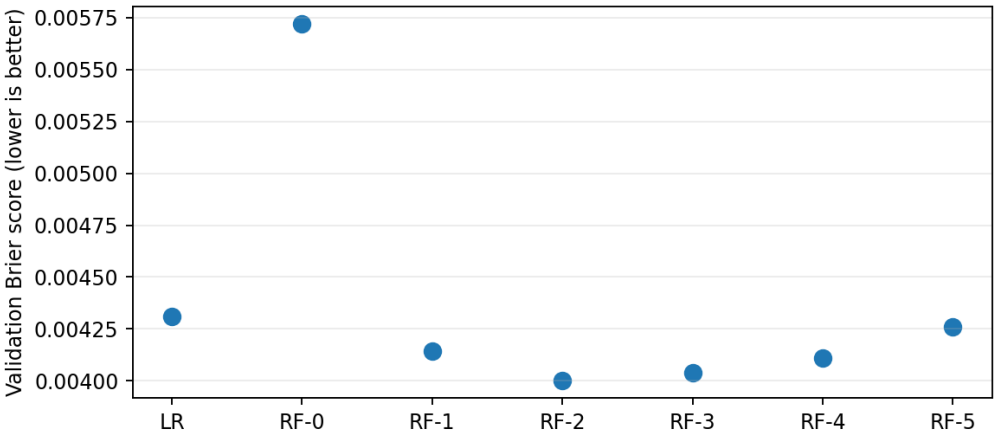


Figure 5: Discrete-model comparison of Brier score; reconstructed from the notebook's validation summary values.

7.4 Why RF-3 was selected, and what was traded off

RF-3 has the highest validation AP and ROC-AUC among the tested configurations. RF-2 has the lowest validation Brier score and log loss. Thus RF-3 is a discrimination-focused selection, *not* a uniform winner on every metric. The regularized RF-3 uses 200 trees, maximum depth 10, minimum leaf size 20, `max_features` = `sqrt`, and random state 42. The unrestricted RF-0 averaged approximately 2,245.59 leaves per tree, versus 106.605 for RF-3. These are empirical complexity summaries, not universal bounds on generalization error.

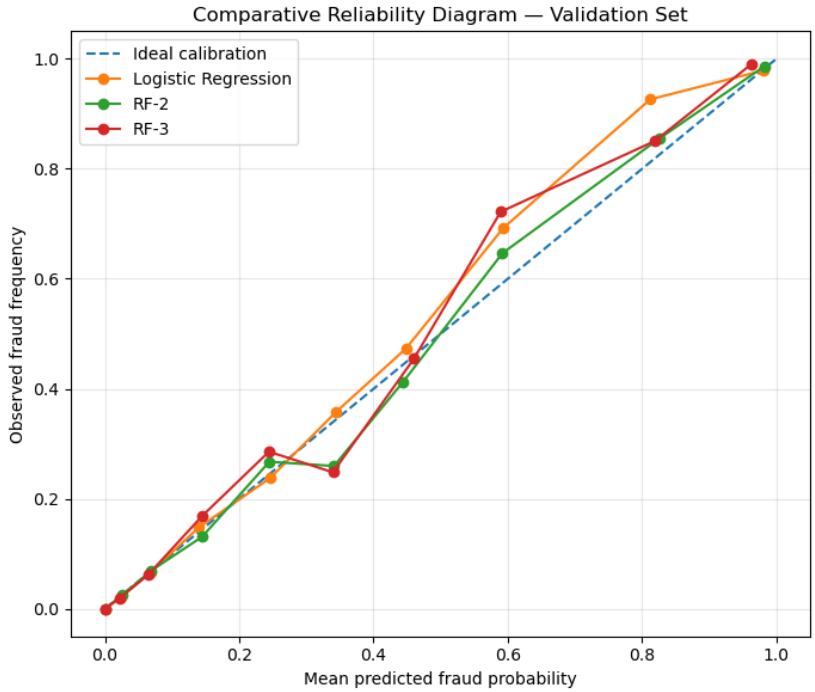


Figure 6: Original comparative validation calibration figure exported from the modeling notebook.

8 Final evaluation on the reserved chronological test partition

8.1 Protocol and predictive results

The test partition consists of simulation steps 153–179: $n = 100{,}117$ transactions, including 1,080 labeled fraud cases and 99,037 non-fraud cases. The model and thresholds were frozen before evaluating this partition.

Metric	RF-3 test result
Average Precision	0.804028
ROC-AUC	0.992570
Brier score	0.003996
Log loss	0.015835
Fraud prevalence	0.010787

8.2 Action-by-outcome analysis

Action	Non-fraud	Fraud	Total
Approve	98,191	200	98,391
Review	825	271	1,096
Escalate	21	609	630
Total	99,037	1,080	100,117

Table 2: Actual final test outcomes under the frozen RF-3 policy.

Define the intervention set $I = \{i : \delta(S_i) \in \{R, E\}\}$. Then

$$\text{FraudCoverage} = \frac{271 + 609}{1080} = \frac{880}{1080} \approx 0.81481, \tag{29}$$

$$\text{InterventionRate} = \frac{1096 + 630}{100117} = \frac{1726}{100117} \approx 0.01724, \tag{30}$$

$$\text{InterventionPrecision} = \frac{880}{1726} \approx 0.50985, \tag{31}$$

$$\text{EscalationPrecision} = \frac{609}{630} \approx 0.96667. \tag{32}$$

There were 846 legitimate interventions $(825 + 21)$ and 200 fraudulent transactions assigned Approve. These counts are *routing outcomes*; they do not establish that fraud was stopped or losses avoided.

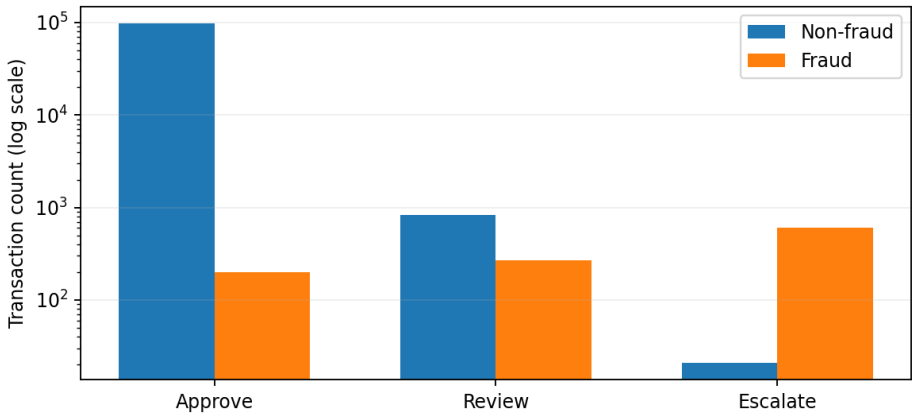


Figure 7: Grouped test action-by-outcome counts (logarithmic vertical axis), reconstructed from the saved test action table.

8.3 Fixed thresholds versus realized workload

Action	Validation count	Test count
Approve	96,398	98,391
Review	1,111	1,096
Escalate	685	630

The same two numerical thresholds were used for both partitions. Validation capacities informed their selection, but the test action counts emerged from the new score distribution. Hence the thresholds do not impose hard workload limits on future batches.

8.4 Hypothetical economic sensitivity

Let $c(a, y)$ represent an assumed cost of action a when the true label is y. A scenario total is

$$C = \sum_{i=1}^{n_{\text{test}}} c(\delta(S_i), Y_i). \tag{33}$$

For the actual test action counts, the notebook's cost convention is

$$C = C_F F_A + C_R N_R + r C_F F_R + C_E(N_E - F_E) + e C_F F_E, \tag{34}$$

where F_A, F_R, F_E are fraud counts in Approve, Review, Escalate, and N_R, N_E are total review and escalation counts. Here r and e are assumed residual fractions of fraud loss after the

corresponding interventions. This expression charges C_E only for legitimate escalations, as implemented in the notebook; it is a stylized convention, not a general operational accounting identity.

Scenario	C_F	C_R	C_E	r	e
A: Review effective	100	1	5	0.10	0.05
B: Review less effective	100	2	5	0.50	0.05
C: Escalation costly	100	2	30	0.20	0.05

For example, Scenario A gives

$$100(200) + 1(1096) + 0.10(100)(271) + 5(21) + 0.05(100)(609) = 26{,}956.$$

The other scenarios follow from the same counts and formula, with their respective assumed parameters. All costs are hypothetical normalized units.

The notebook explored three distinct hypothetical cost scenarios; their reported totals were 26,956 (A: Review effective), 38,892 (B: Review less effective), and 31,287 (C: Escalation costly), in normalized units. These numbers are conditional on the notebook's assumptions and *are not* measured financial savings.

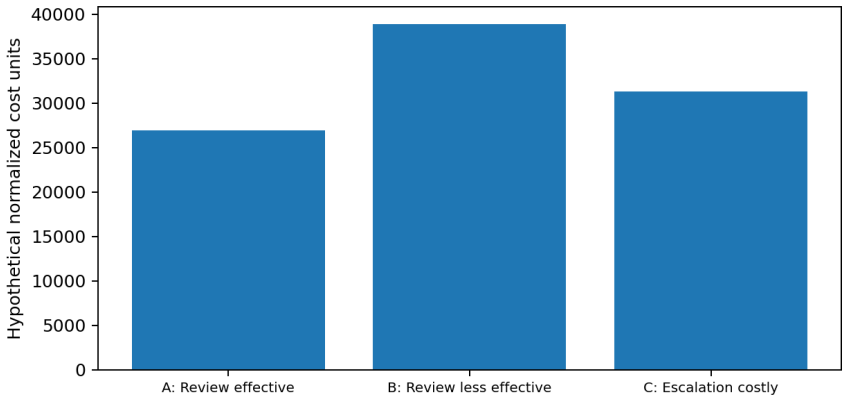


Figure 8: Hypothetical scenario totals from the final test evaluation; chart reconstructed from reported values.

8.5 What the test results can and cannot support

The reserved test provides evidence of discrimination and operational prioritization under the synthetic BankSim distribution. It does not establish external validity, real-world probability calibration, causal intervention effectiveness, or live banking readiness. Repeatedly selecting policies based on test performance would invalidate its role as a reserved evaluation partition.

9 Reproducible inference, monitoring, and limitations

The saved scikit-learn pipeline is `models/rf3_pipeline.joblib`; the separate frozen policy is `models/rf3_policy.json`. The local batch program `src/predict.py` accepts a CSV containing `amount` and `category`, validates the input, predicts scores, and assigns actions according to Equation (19). Five sample transactions were processed; the standalone scores agreed with the original notebook scores within strict floating-point tolerances and their actions matched exactly. This is a five-record reproducibility check, not a comprehensive software certification.

9.1 Monitoring framework

Input-quality monitoring should detect missing or invalid values and unfamiliar categories. Distribution monitoring should compare incoming amount and category frequencies with reference data. Score and workload monitoring can be immediate; AP, calibration, and fraud coverage require delayed, reliable labels. Threshold governance should distinguish model changes from policy changes and record each version.

9.2 Educational synthesis

A complete pipeline separates four questions: *What is the target conditional probability?* (statistical model), *How well does the fitted score rank and quantify risk?* (validation), *Which actions are justified by assumptions and constraints?* (decision theory), and *What actually happens after intervention?* (causal and operational evaluation). Conflating these questions leads to overconfident claims.

10 Principal findings, contributions, limitations, and conclusions

10.1 What the experiments established

The supervised binary formulation was appropriate to the labeled BankSim response, while the three-action policy was correctly treated as a separate downstream decision layer. On chronological validation, RF-3 achieved the highest Average Precision (0.807737) and ROC-AUC (0.992853) among the seven evaluated models (Logistic Regression and RF-0–RF-5). RF-2 instead had the lowest Brier score (0.004003) and log loss (0.015751). Thus RF-3 was selected for *ranking discrimination*, not because it dominated every measure of probability quality.

With the RF-3 model and thresholds frozen, the reserved chronological test yielded Average Precision 0.804028, ROC-AUC 0.992570, Brier score 0.003996, and log loss 0.015835. The policy routed 880 of 1,080 fraud-labeled transactions (81.48%) to Review or Escalate, while assigning 1,726 of 100,117 transactions (1.72%) to those actions. It also assigned 200 labeled fraud transactions to Approve and 846 non-fraud transactions to intervention. These trade-offs are essential to interpreting the result: the policy neither captures every fraud case nor avoids all unnecessary interventions.

10.2 What this project contributes

The principal contribution is a reproducible, documented *case study*, not a new ML method: a CRISP-DM-aligned problem formulation; data-type-aware, training-only preprocessing; an interpretable baseline and controlled forest comparison; discrimination and probability-quality assessment; validation-derived, fixed three-action thresholds; reserved test evaluation; and a standalone inference interface. The mathematical derivations and project outputs distinguish literature-supported principles, project-specific design choices, and empirical findings.

10.3 Limits and next steps

BankSim is synthetic and may have simulation artifacts; the feature set is deliberately restricted to amount and category. Temporal ordering of simulation steps does not establish real-world temporal transferability. Reliability plots and proper scoring rules on BankSim do not prove calibration for real banking transactions. The intervention outcomes are simulated *routing decisions*, not observed prevention effects. The cost scenarios are assumption-dependent normalized quantities, not verified financial savings. Future evaluation should use a separate dataset or operationally representative data, assess drift, leakage, calibration, subgroup performance, and delayed labels, and estimate genuine intervention effects and costs before any live deployment.

Conclusion. The study demonstrates that a carefully evaluated, limited-feature supervised classifier can support selective fraud-risk prioritization *within the BankSim simulation*. Its broader lesson is methodological: the problem definition, data types, split design, model comparison, probability assessment, decision policy, and final test must be justified separately and then connected in one reproducible workflow.

11 Questions for mathematical understanding and independent study

1. Why is the response binary when the platform has three possible actions? Distinguish observed labels, model probabilities, and policy actions.
2. Why use a logistic rather than a linear probability model? Derive the logit and logistic inverse.
3. Where do cross-entropy and maximum likelihood come from? Derive them from the Bernoulli probability mass function.
4. Why is logistic loss convex? Prove that the Hessian is positive semidefinite.
5. What does β_1 mean when amount is standardized? Explain odds ratios and scaling.
6. Why can all one-hot indicators plus an intercept create collinearity? Explain how regularization affects coefficients.
7. What is learned by `fit`, and what must not be learned from validation data?
8. What do AP and ROC-AUC measure? Why is AP not simply trapezoidal PR-AUC?
9. Why can AP be high while recall at threshold 0.5 is only 58.24%?
10. Why should the final test set remain untouched until development decisions are frozen?

Reference notes

ISLP notation and classification framing are used throughout [7]. The algebraic proofs in this document are worked derivations of standard results, not claims of original research. The numerical findings are taken from the executed BankSim notebook; references support the methods, not the specific BankSim outputs. The approved Python excerpts are taken from `notebooks/03_modeling.ipynb` (Sections 4.2, 4.3.1, and 4.3.2), preserving the original statements and parameters. They are excerpts of sequential notebook cells, not a complete standalone script. The project environment specifies scikit-learn 1.5.1; the API links below use the 1.5 documentation family rather than the evolving stable branch. No model was retrained to produce this document.

References

[1] J. O. Berger, *Statistical Decision Theory and Bayesian Analysis*, 2nd ed., Springer, 1985. https://doi.org/10.1007/978-1-4757-4286-2.

[2] L. Breiman, "Random forests," *Machine Learning*, 45, pp. 5–32, 2001. https://doi.org/10.1023/A:1010933404324.

[3] P. Chapman et al., *CRISP-DM 1.0: Step-by-Step Data Mining Guide*, CRISP-DM Consortium, 2000.

[4] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, 27(8), pp. 861–874, 2006. https://doi.org/10.1016/j.patrec.2005.10.010.

[5] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, 102(477), pp. 359–378, 2007. https://doi.org/10.1198/016214506000001437.

[6] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed., Springer, 2009. https://doi.org/10.1007/978-0-387-84858-7.

[7] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, Springer, 2023. https://doi.org/10.1007/978-3-031-38747-0.

[8] E. A. López-Rojas, A. Elmir, and S. Axelsson, "BankSim: A bank payments simulator for fraud detection research," *Proceedings of the 26th European Modeling and Simulation Symposium (EMSS)*, 2014.

[9] P. McCullagh and J. A. Nelder, *Generalized Linear Models*, 2nd ed., Chapman & Hall, 1989.

[10] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," *Proceedings of the 22nd International Conference on Machine Learning*, pp. 625–632, 2005. https://doi.org/10.1145/1102351.1102430.

[11] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006. https://doi.org/10.1007/978-0-387-40065-5.

[12] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, 10(3), e0118432, 2015. https://doi.org/10.1371/journal.pone.0118432.

[13] scikit-learn developers, *average_precision_score documentation*, version 1.5 documentation. https://scikit-learn.org/1.5/modules/generated/sklearn.metrics.average_precision_score.html.

[14] scikit-learn developers, *LogisticRegression documentation*, version 1.5 documentation. https://scikit-learn.org/1.5/modules/generated/sklearn.linear_model.LogisticRegression.html.

[15] scikit-learn developers, *OneHotEncoder documentation*, version 1.5 documentation. https://scikit-learn.org/1.5/modules/generated/sklearn.preprocessing.OneHotEncoder.html.

[16] scikit-learn developers, *StandardScaler documentation*, version 1.5 documentation. https://scikit-learn.org/1.5/modules/generated/sklearn.preprocessing.StandardScaler.html.